StudySync — Peer Matching & Collaborative Study Group Platform

Requirements Modeling: Use Cases & Master User Stories Catalog

Course: Software Engineering & Project Management

Project Group: Group 15 (Section F)

Target System: StudySync Web Platform (React 18 + Node.js + Express + PostgreSQL 3NF)

Date of Release: October 16, 2026

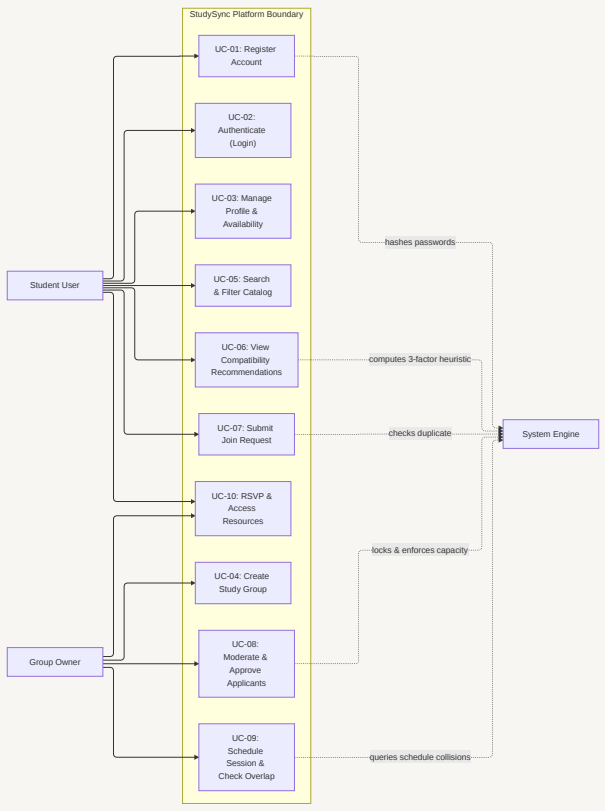
Document Standard: Agile User Stories & IEEE Use-Case Specification

1. Overview & Scope

This document provides the formal software requirements models for **StudySync**. In accordance with the course evaluation guidelines (Section 3.A, 3.C, and 4), this artifact presents both:

1. **Behavioral Use Cases:** Complete textual use-case specifications capturing actor goals, preconditions, normal flows, exception branches, and postconditions.
2. **Agile User Stories:** Detailed Agile User Stories formatted as *"As a... I want... So that..."*, assigned Fibonacci story points, MoSCoW priorities, and structured **Given-When-Then** acceptance criteria (Definition of Done).

2. Master Use-Case Diagram



3. Actor Catalog & System Boundary

Actor Name	Type	Description
Student User	Human (Primary)	Any authenticated university student seeking study partners, managing availability, searching course-aligned groups, applying for membership, and RSVPing to study sessions.
Group Owner	Human (Primary)	An authenticated student who created a study group. Holds exclusive administrative rights to edit group details, review applicants, approve/reject memberships, schedule sessions, and upload resources.

Actor Name	Type	Description
System Engine	Automated (Secondary)	Backend server background validation and computation services: password hashing (bcrypt), token issuance (JWT), schema validation (Zod), heuristic scoring math, and transactional capacity locking (`SELECT FOR UPDATE`).

4. Detailed Use-Case Specifications

UC-01: Student Registration & Account Onboarding

- **Primary Actor:** Unauthenticated Student User
- **Secondary Actor:** System Engine
- **Preconditions:** User has a valid university email address (`@university.edu`) and is not logged in.
- **Trigger:** Student clicks "Create Account" on the navigation bar.
- **Main Success Scenario:**
  1. Student fills in full name, institutional email, and password.
  2. System validates input against Zod schema (checks email format and password complexity: ≥ 8 chars, uppercase, digit, symbol).
  3. System queries the `users` table to ensure email uniqueness.
  4. System hashes the password via `bcrypt` (10 salt rounds).
  5. System inserts a new record into `users` and creates an associated blank profile in `user_profiles`.
  6. System issues a signed 24-hour JWT token.
  7. Student is redirected to the Profile Setup onboarding screen.
- **Extensions / Exceptions:**
  - *2a. Password Complexity Failure:* System displays inline validation error messages.
  - *3a. Duplicate Email:* System returns HTTP 409 Conflict ("Email is already registered").

UC-02: User Authentication & JWT Session Issuance

- **Primary Actor:** Registered Student
- **Secondary Actor:** System Engine
- **Preconditions:** Student has a registered active account in `users`.
- **Main Success Scenario:**
  1. Student submits institutional email and password via login form.
  2. System retrieves the user record by email.
  3. System verifies credentials using timing-safe `bcrypt.compare`.
  4. System generates a signed JWT bearer token containing `{ userId, email, role }` with 24-hour expiration.
  5. System returns HTTP 200 OK with the token and user metadata.
  6. Frontend stores the token in `localStorage` and routes student to the Groups Dashboard.
- **Extensions / Exceptions:**
  - *2a. Non-existent Email:* System returns HTTP 401 Unauthorized (`INVALID_CREDENTIALS`).
  - *3a. Invalid Password:* System returns HTTP 401 Unauthorized (`INVALID_CREDENTIALS`).

UC-03: Manage Academic Profile & Recurring Weekly Availability

- **Primary Actor:** Authenticated Student User

- **Preconditions:** Student holds an active JWT session.
- **Main Success Scenario:**
  1. Student opens "Profile Settings".
  2. Student inputs major, academic year, personal bio, and target learning goals.
  3. Student specifies recurring weekly free time slots by adding day-of-week (0–6), start time, and end time.
  4. System validates that $\text{endTime} > \text{startTime}$.
  5. System serializes availability as a structured JSONB array and updates `user_profiles`.
  6. System displays a success toast notification.
- **Extensions / Exceptions:**
  - *4a. Inverted Time Range:* System blocks submission with error ("End time must be after start time").

UC-04: Study Group Creation with Capacity Bounds

- **Primary Actor:** Authenticated Student (Becomes Group Owner)
- **Preconditions:** Student is logged in.
- **Main Success Scenario:**
  1. Student clicks "Create Study Group" in the group catalog.
  2. Student enters: group name, description, selects course subject from catalog, sets member capacity cap ($2 \leq n \leq 50$), and selects visibility (`public` or `private`).
  3. System validates parameters and ensures subject UUID exists.
  4. System inserts record into `study_groups`.
  5. System automatically inserts a record into `group_memberships` assigning creator role `'owner'` with status `'accepted'`.
  6. System navigates the user to the newly created Group Management view.
- **Extensions / Exceptions:**
  - *2a. Member Capacity Outside Range:* If $n < 2$ or $n > 50$, system returns HTTP 422 Bad Request.

UC-05: Browse, Search & Filter Study Group Catalog

- **Primary Actor:** Student User
- **Preconditions:** Platform database contains published study groups.
- **Main Success Scenario:**
  1. Student navigates to "Explore Groups".
  2. Student applies optional search criteria: search keywords, course subject filter, and open capacity toggle.
  3. System queries `study_groups` joined with `subjects` and `group_memberships`.
  4. System computes active accepted member count for each group in real time.
  5. System returns a paginated list of matching groups with pagination metadata in under 200ms.
  6. Frontend renders interactive group cards displaying subject tag, owner name, description, and capacity badges.

UC-06: View Explainable Algorithmic Compatibility Recommendations

- **Primary Actor:** Authenticated Student User
- **Secondary Actor:** System Engine

- **Preconditions:** Student has configured their profile, preferred subjects, and weekly availability slots.
- **Main Success Scenario:**
  1. Student navigates to the "Recommendations" page.
  2. System fetches student profile and availability from `user_profiles`.
  3. System retrieves active public study groups.
  4. System executes the 3-Factor Compatibility Algorithm:

$$\text{Score} = (S_{\text{subject}} \times 0.40) + (S_{\text{schedule}} \times 0.40) + (S_{\text{goals}} \times 0.20)$$

  5. System sorts groups in descending order of compatibility score.
  6. Frontend displays recommended groups with an explainable breakdown modal (Subject Match %, Schedule Overlap minutes, Shared Learning Goals).

UC-07: Submit Membership Join Request with Motivation Note

- **Primary Actor:** Authenticated Student (Applicant)
- **Secondary Actor:** System Engine
- **Preconditions:** Group visibility is public, and current accepted count $< \text{max_members}$.
- **Main Success Scenario:**
  1. Student opens a Group Detail view and clicks "Request to Join".
  2. Student inputs an optional motivation note explaining their study goals.
  3. System verifies that student does not already have an active or pending membership in this group.
  4. System inserts a record into `group_memberships` with `role = 'member'`, `status = 'pending'`, and `request_note`.
  5. System returns HTTP 201 Created.
  6. Frontend updates button state to "Application Pending".
- **Extensions / Exceptions:**
  - *3a. Duplicate Request:* System returns HTTP 409 Conflict ("You have already applied or are a member").

UC-08: Review, Moderate & Approve Group Membership Applications

- **Primary Actor:** Group Owner
- **Secondary Actor:** System Engine
- **Preconditions:** Group has pending applicant records in `group_memberships`.
- **Main Success Scenario:**
  1. Group Owner navigates to "Owner Review Panel" on their group page.
  2. System displays list of pending applicants, their profile summaries, applicant notes, and compatibility scores.
  3. Group Owner clicks "Approve".
  4. System initiates an atomic database transaction with row-level locking (`SELECT ... FOR UPDATE`).
  5. System verifies that current accepted members count is strictly less than `max_members`.
  6. System updates membership status to `'accepted'` and sets `joined_at = NOW()`.
  7. System increments active member count and commits transaction.
  8. Frontend updates applicant badge to "Accepted".
- **Extensions / Exceptions:**

- *5a. Group Capacity Reached:* If group reached capacity concurrently, system aborts approval, rolls back, and returns HTTP 400 Bad Request (`GROUP_CAPACITY_REACHED`).
- *3b. Owner Rejects Applicant:* Status is updated to `'rejected'`; applicant note is archived.

UC-09: Schedule Study Session with Collision Detection

- **Primary Actor:** Group Owner
- **Secondary Actor:** System Engine
- **Preconditions:** Study group is active and user is owner.
- **Main Success Scenario:**
  1. Group Owner opens "Schedule Session" modal.
  2. Owner inputs: title, description, start timestamp, end timestamp, location type (`online` or `in_person`), and meeting URL or classroom venue.
  3. System validates that $\text{endTime} > \text{startTime} > \text{NOW}()$.
  4. System executes overlap query across existing scheduled sessions for all enrolled members.
  5. If collisions exist, system presents an advisory warning banner detailing conflicting member names.
  6. Owner confirms session creation.
  7. System inserts record into `study_sessions` and generates `session_attendees` records for all group members with default status `'tentative'`.

UC-10: Session RSVP, Calendar Navigation & Resource Access

- **Primary Actor:** Enrolled Group Member
- **Preconditions:** Student holds an accepted membership in the study group.
- **Main Success Scenario:**
  1. Student navigates to "My Schedule" calendar view.
  2. System retrieves all upcoming sessions across the student's enrolled groups.
  3. Student selects a session and toggles their attendance RSVP (`attending`, `declined`, `tentative`).
  4. System updates `session_attendees` record and updates live attendee roster.
  5. Student opens the "Resources" tab and downloads shared lecture notes, links, and study guides.
- **Extensions / Exceptions:**
  - *5a. Unauthorized Access Attempt:* If a non-member attempts to access private group resources, system returns HTTP 403 Forbidden.

---

5. Master Product Backlog: User Stories with Gherkin Acceptance Criteria

US-01: Student Registration & Validation

- **Role:** As a prospective university student,
- **Action:** I want to register for StudySync using my institutional email and a secure password,
- **Benefit:** So that I can establish a verified academic identity on the platform.
- **Story Points:** 5 | **Priority:** Must Have | **Sprint:** Sprint 1 (Increment 1)
- **Acceptance Criteria:**

    - **Scenario 1: Successful Registration**
        - **Given** an unregistered email `student@university.edu` and valid password `Passw0rd!`,
        - **When** I submit the registration form,
        - **Then** the system creates my account in `users`, hashes my password with bcrypt (10 rounds), returns HTTP 201 Created, and issues a 24-hour JWT token.
    - **Scenario 2: Duplicate Email Rejection**
        - **Given** an email address that already exists in the system,
        - **When** a user attempts to register with that email,
        - **Then** the system rejects registration with HTTP 409 Conflict.

US-02: JWT Authentication & Session Retention

- **Role:** As a registered student,
- **Action:** I want to log in with my email and password,
- **Benefit:** So that I can securely access my private study groups and personal schedule.
- **Story Points:** 5 | **Priority:** Must Have | **Sprint:** Sprint 1 (Increment 1)
- **Acceptance Criteria:**
    - **Scenario 1: Valid Login**
        - **Given** valid registered credentials,
        - **When** I submit the login form,
        - **Then** I receive a signed JWT token and user profile object, and am redirected to the Dashboard.
    - **Scenario 2: Incorrect Password Handling**
        - **Given** a registered email with an invalid password,
        - **When** I submit the login form,
        - **Then** the system responds with HTTP 401 Unauthorized without exposing whether the email exists.

US-03: Profile Setup & Academic Major

- **Role:** As an authenticated student,
- **Action:** I want to save my academic major, academic year, bio, and learning goals,
- **Benefit:** So that other students can understand my academic focus when forming groups.
- **Story Points:** 3 | **Priority:** Must Have | **Sprint:** Sprint 1 (Increment 1)
- **Acceptance Criteria:**
    - **Scenario 1: Profile Update**
        - **Given** an authenticated student session,
        - **When** I update my profile with major `Software Engineering` and goals `["Pass Exam", "Raft Consensus"]`,
        - **Then** the data is saved in `user_profiles` and reflected in `/api/v1/auth/me`.

US-04: Weekly Availability Slots in JSONB

- **Role:** As a student,
- **Action:** I want to specify recurring weekly time slots when I am free to study,
- **Benefit:** So that StudySync can match me with peers who have overlapping schedules.
- **Story Points:** 5 | **Priority:** Must Have | **Sprint:** Sprint 1 (Increment 1)
- **Acceptance Criteria:**

- **Scenario 1: Availability Persistence**
  - **Given** a weekly slot array `[{"dayOfWeek": 2, "startTime": "14:00", "endTime": "16:00"}]`,
  - **When** I save my availability,
  - **Then** the system verifies `endTime > startTime` and saves the JSONB payload in `user_profiles`.

US-05: Course-Centric Study Group Creation

- **Role:** As an authenticated student,
- **Action:** I want to create a new study group associated with a specific university subject and member cap,
- **Benefit:** So that I can lead collaborative study sessions for my course.
- **Story Points:** 5 | **Priority:** Must Have | **Sprint:** Sprint 1 (Increment 1)
- **Acceptance Criteria:**
  - **Scenario 1: Group Creation**
    - **Given** a valid group name, description, subject UUID, and capacity 6,
    - **When** I submit the group creation form,
    - **Then** a `study_groups` record is created, and I am automatically assigned membership role `'owner'` with status `'accepted'`.
  - **Scenario 2: Boundary Validation**
    - **Given** a member capacity of 1 or 55,
    - **When** I submit the form,
    - **Then** the system rejects the submission with HTTP 422 Bad Request.

US-06: Study Group Directory Search & Filters

- **Role:** As a student,
- **Action:** I want to search and filter study groups by subject code, department, and keywords,
- **Benefit:** So that I can discover study groups relevant to my enrolled courses.
- **Story Points:** 5 | **Priority:** Must Have | **Sprint:** Sprint 1 (Increment 1)
- **Acceptance Criteria:**
  - **Scenario 1: Catalog Search Execution**
    - **Given** 10 active study groups in the database,
    - **When** I query `GET /api/v1/groups?search=Distributed&subjectId=CS301`,
    - **Then** the response returns matching groups with live member counts in under 200ms.

US-07: Join Request Submission with Applicant Note

- **Role:** As a student,
- **Action:** I want to submit a join request to a public study group along with an introductory note,
- **Benefit:** So that the group owner can evaluate my motivation before admitting me.
- **Story Points:** 3 | **Priority:** Must Have | **Sprint:** Sprint 2 (Increment 2)
- **Acceptance Criteria:**
  - **Scenario 1: Submitting Request**
    - **Given** a public group with open seats,
    - **When** I send a request with note `"Looking forward to studying algorithms"`,

        - **Then** a `group_memberships` record is created with `status = 'pending'`.
    - **Scenario 2: Duplicate Request Guard**
        - **Given** an existing pending request,
        - **When** I attempt to send a second join request,
        - **Then** the request is rejected with HTTP 409 Conflict.

US-08: Owner Applicant Review Panel & Approval

- **Role:** As a study group owner,
- **Action:** I want to review pending applicants and approve or reject them,
- **Benefit:** So that I can curate group membership while respecting the group capacity ceiling.
- **Story Points:** 5 | **Priority:** Must Have | **Sprint:** Sprint 2 (Increment 2)
- **Acceptance Criteria:**
    - **Scenario 1: Approving Member**
        - **Given** a pending applicant in a group with available capacity,
        - **When** the owner clicks "Approve",
        - **Then** the status updates to `'accepted'`, the member count increments, and `joined_at` is timestamped.
    - **Scenario 2: Capacity Boundary Lock**
        - **Given** a group at full capacity ($n = \text{max_members}$),
        - **When** the owner attempts to approve another applicant,
        - **Then** the system returns HTTP 400 Bad Request (`GROUP_CAPACITY_REACHED`) and prevents over-enrollment.

US-09: Algorithmic Peer Compatibility Calculation

- **Role:** As a student,
- **Action:** I want the system to compute a multi-factor compatibility score between me and study groups,
- **Benefit:** So that I can immediately identify groups that fit my courses and schedule.
- **Story Points:** 8 | **Priority:** Should Have | **Sprint:** Sprint 2 (Increment 2)
- **Acceptance Criteria:**
    - **Scenario 1: High Compatibility Group**
        - **Given** a student and a group with identical subject, shared free slots on Tuesday, and matching goals,
        - **When** the compatibility algorithm executes,
        - **Then** the computed score is $\geq 80\%$ with an explainable criteria breakdown.

US-10: Ranked Group Recommendations Feed

- **Role:** As a student,
- **Action:** I want to browse a recommendations feed displaying groups ranked by compatibility score,
- **Benefit:** So that I spend less time searching and more time studying.
- **Story Points:** 5 | **Priority:** Should Have | **Sprint:** Sprint 2 (Increment 2)
- **Acceptance Criteria:**
    - **Scenario 1: Ranked Recommendations**

        - **Given** multiple active study groups,
        - **When** I view `/api/v1/recommendations/groups`,
        - **Then** groups are returned in descending order of compatibility score.

US-11: Study Session Scheduling

- **Role:** As a group owner,
- **Action:** I want to schedule study sessions with dates, times, and meeting locations,
- **Benefit:** So that group members can plan their calendar and prepare in advance.
- **Story Points:** 5 | **Priority:** Must Have | **Sprint:** Sprint 3 (Increment 3)
- **Acceptance Criteria:**
    - **Scenario 1: Valid Session Creation**
        - **Given** a future time window where `endTime > startTime`,
        - **When** the owner creates a session,
        - **Then** a `study_sessions` record is created, and attendees are initialized to `'tentative'`.

US-12: Session Schedule Overlap & Collision Guard

- **Role:** As a system,
- **Action:** I want to detect schedule collisions across enrolled members before confirming a session,
- **Benefit:** So that organizers avoid scheduling meetings when key members have conflicts.
- **Story Points:** 5 | **Priority:** Should Have | **Sprint:** Sprint 3 (Increment 3)
- **Acceptance Criteria:**
    - **Scenario 1: Conflict Detection**
        - **Given** a member who already has another session scheduled during the proposed window,
        - **When** the owner proposes the session,
        - **Then** the system returns an advisory alert displaying the conflicting member's name and overlapping time.

US-13: Interactive Personal & Group Calendar View

- **Role:** As a student,
- **Action:** I want to view all my upcoming study sessions on an interactive calendar,
- **Benefit:** So that I can visualize my weekly academic commitments at a glance.
- **Story Points:** 5 | **Priority:** Must Have | **Sprint:** Sprint 3 (Increment 3)
- **Acceptance Criteria:**
    - **Scenario 1: Calendar Rendering**
        - **Given** scheduled sessions across 3 enrolled groups,
        - **When** I open the Calendar view,
        - **Then** sessions are displayed with color-coded status badges and month/week/day filters.

US-14: RSVP Tracking & Verified Attendance Check-In

- **Role:** As an enrolled member,
- **Action:** I want to submit an RSVP (`attending`, `declined`, `tentative`) and check in during active meetings,
- **Benefit:** So that the group leader knows who will attend and tracks participation.

- **Story Points:** 3 | **Priority:** Should Have | **Sprint:** Sprint 3 (Increment 3)
- **Acceptance Criteria:**
  - **Scenario 1: RSVP Update**
    - **Given** an upcoming session,
    - **When** I toggle my RSVP to `'attending'`,
    - **Then** my attendee status is updated and reflected in the live attendee roster.

US-15: Centralized Shared Academic Study Resources

- **Role:** As a group member,
- **Action:** I want to upload and share study documents, links, and code notes within my group,
- **Benefit:** So that all study materials are organized in one secure repository.
- **Story Points:** 5 | **Priority:** Should Have | **Sprint:** Sprint 3 (Increment 3)
- **Acceptance Criteria:**
  - **Scenario 1: Member Access**
    - **Given** an authenticated member of Group A,
    - **When** they access Group A's resources,
    - **Then** they can view and download all shared materials.
  - **Scenario 2: Non-Member Access Block**
    - **Given** an authenticated user who is not a member of Group A,
    - **When** they attempt to access Group A's resources,
    - **Then** the request is rejected with HTTP 403 Forbidden.

---

6. Traceability to Functional Requirements

User Story ID	Associated Functional Requirement	Target Sprint	Primary Test Case ID	Mid-Sem Evaluation Status
US-01	`FR-001`, `FR-003`	Sprint 1	`TC-001`, `TC-002`, `TC-017`	**PASS**
US-02	`FR-002`	Sprint 1	`TC-003`, `TC-004`	**PASS**
US-03	`FR-004`	Sprint 1	`TC-005`	**PASS**
US-04	`FR-005`	Sprint 1	`TC-005`	**PASS**
US-05	`FR-006`, `FR-010`	Sprint 1	`TC-006`, `TC-007`	**PASS**
US-06	`FR-007`	Sprint 1	`TC-008`	**PASS**
US-07	`FR-008`	Sprint 2	`TC-009`	**PASS**
US-08	`FR-009`, `FR-010`	Sprint 2	`TC-010`	**PASS**
US-09	`FR-011`	Sprint 2	`TC-011`, `TC-012`	**PASS**
US-10	`FR-012`	Sprint 2	`TC-012`	**PASS**
US-11	`FR-013`, `FR-014`	Sprint 3	`TC-013`, `TC-014`	PLANNED (Sprint 3)
US-12	`FR-015`	Sprint 3	`TC-014`	PLANNED (Sprint 3)
US-13	`FR-016`	Sprint 3	`TC-013`	PLANNED (Sprint 3)
US-14	`FR-017`, `FR-018`	Sprint 3	`TC-015`	PLANNED (Sprint 3)
US-15	`FR-019`, `FR-020`	Sprint 3	`TC-016`	PLANNED (Sprint 3)